

BOOTCAMP VIBE CODING • COMPÉTITION

DU BESOIN À LA RELEASE

Pourquoi la méthode reste reine, même quand l'IA code à votre place.



Suisse • SSVAR • Prof. M. J-P. Sangaré

20

MIN

Vibe coder vite... pour livrer quoi ?



pour générer un prototype qui « marche » à l'écran

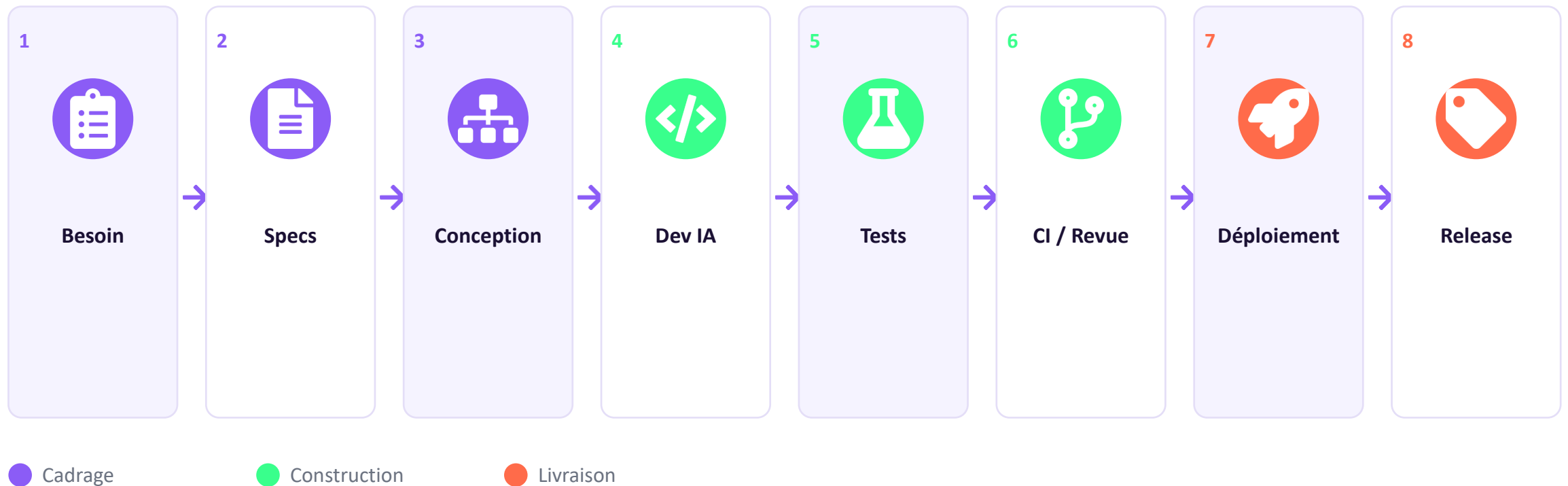


pour déboguer, sécuriser et livrer ce même code sans méthode

Vibe coding = vitesse d'exécution. Sans les étapes du cycle de vie logiciel, cette vitesse se transforme en dette technique instantanée — et c'est exactement ce que le jury va sanctionner.

8 étapes, du besoin à la Release

Chaque phase de la compétition sera évaluée à l'aune de ce pipeline complet.



01



Expression de besoin

Comprendre le problème avant d'écrire une ligne de code.

- Qui est l'utilisateur final ? Quel problème concret résout-on ?
- Rédiger des user stories : « En tant que..., je veux..., afin de... »
- Définir des critères d'acceptation mesurables et vérifiables
- Identifier les parties prenantes et les contraintes (temps, budget, tech)



Le jury pose souvent la question « pourquoi cette fonctionnalité ? » avant « comment l'avez-vous codée ? ».

02



Spécifications fonctionnelles & techniques

Traduire le besoin en exigences précises et non ambiguës.



Spécifier le périmètre fonctionnel (ce qui est IN / OUT du MVP)



Choisir la stack technique et justifier ce choix



Définir les contraintes non-fonctionnelles : sécurité, performance, accessibilité



Prioriser avec un MoSCoW ou un backlog ordonné



Un prompt bien écrit part toujours d'une spécification claire — le flou en amont produit du flou dans le code généré.

03



Conception & architecture

Là où le vibe coding doit s'arrêter et la réflexion commencer.

- Modéliser les données et les flux avant de générer du code
- Découper en modules / composants avec des responsabilités claires
- Anticiper la scalabilité et la maintenabilité
- Documenter les décisions d'architecture (mini ADR)



L'IA excelle à écrire du code, pas à décider de l'architecture : cette étape reste 100% humaine.

04



Développement assisté par IA

Le vibe coding entre en scène — mais toujours guidé.



Prompting itératif : petites unités de travail, vérifiées à chaque étape



Relire et comprendre le code généré avant de l'accepter



Respecter les conventions du projet (nommage, structure, style)



Versionner fréquemment avec des commits atomiques et clairs



Vibe coder sans lire le code, c'est signer un contrat sans le lire.

05



Tests

Prouver que le logiciel fait ce qu'il est censé faire.

● Tests unitaires sur les fonctions et composants critiques

● Tests d'intégration entre modules et services

● Recette utilisateur : valider par rapport aux critères d'acceptation

● Demander à l'IA de générer des cas de test, puis les challenger



Un code qui « marche à la démo » n'est pas un code testé — le jury essaiera de le casser.

06



Intégration continue & revue de code

Fusionner sans casser ce qui fonctionne déjà.

- Pipeline CI : build + tests automatiques à chaque push
- Pull requests avec revue de code, même en solo (auto-revue à froid)
- Linting et analyse statique pour la qualité du code
- Gérer les conflits de fusion et l'historique Git proprement



Une CI verte n'est pas un détail technique : c'est la preuve que le projet est livrable.

07



Déploiement

Faire passer le projet de « ça marche chez moi » à « ça marche pour tous ».



Préparer l'environnement de production (config, variables, secrets)



Automatiser le déploiement (script, CI/CD, conteneurs)



Prévoir un plan de rollback en cas d'échec



Vérifier la performance et la sécurité en conditions réelles



Le jour J de la compétition, un déploiement fragile peut ruiner un excellent code.

08



Release & versioning

Livrer une version identifiable, traçable et « definition of done ».

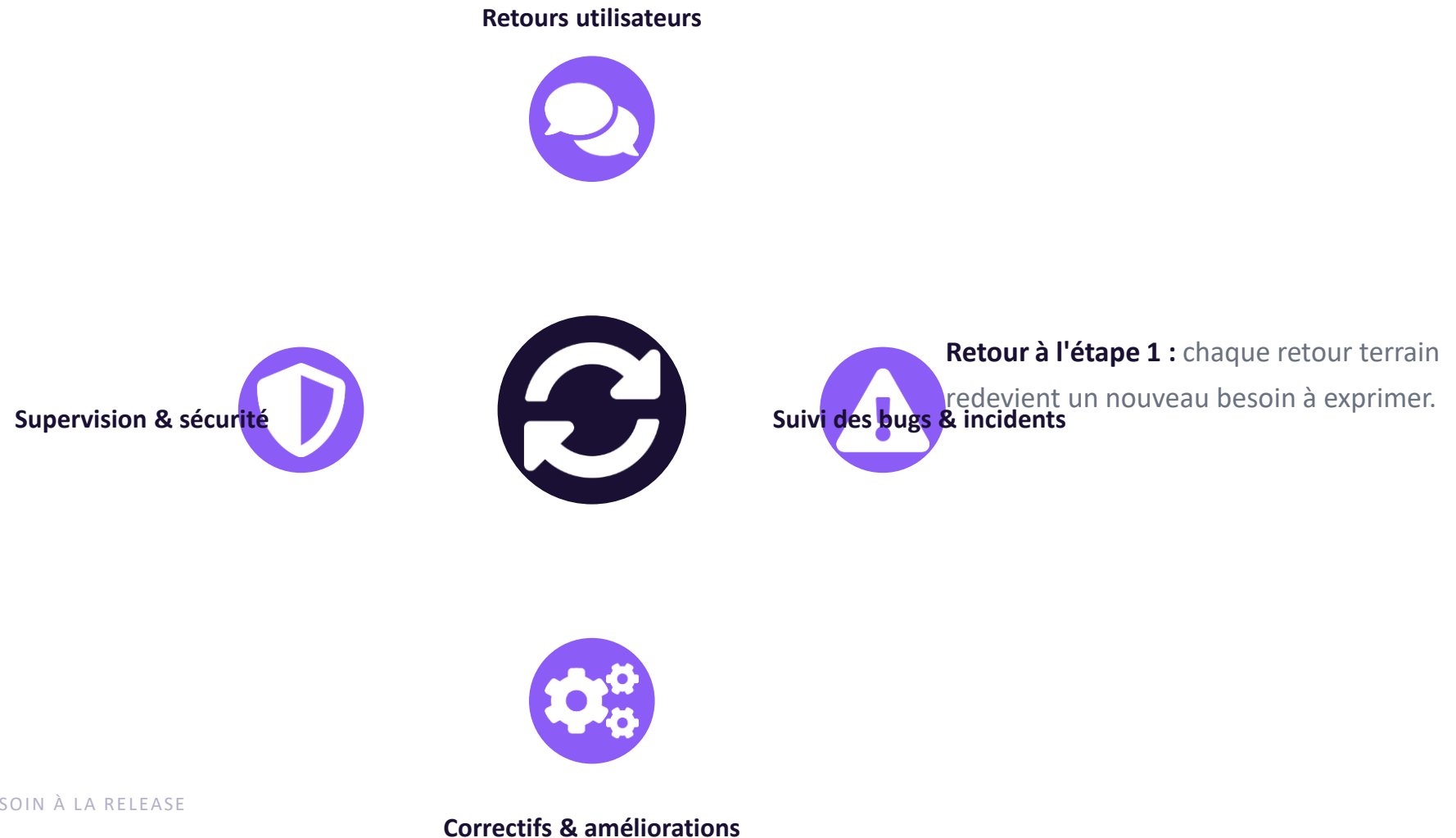
- Versionner selon le semantic versioning (MAJOR.MINOR.PATCH)
- Rédiger un changelog clair pour le jury et les utilisateurs
- Vérifier la « definition of done » : tout critère d'acceptation validé
- Préparer la documentation de livraison (README, guide d'usage)



Une Release, c'est un engagement : « voici ce qui fonctionne, à cette date, dans ces conditions ».

La boucle de feedback & maintenance

Un logiciel livré est un logiciel qui recommence à apprendre de ses utilisateurs.



Checklist auto-évaluation express

8 questions à vous poser avant de soumettre — une par étape du pipeline.



Puis-je résumer le besoin en une phrase claire ?



Mes critères d'acceptation sont-ils écrits et vérifiables ?



Mon architecture est-elle documentée, même sommairement ?



Ai-je relu et compris tout le code généré par l'IA ?



Mes fonctionnalités critiques sont-elles testées ?



Ma CI passe-t-elle au vert avant la démo ?



Mon déploiement a-t-il un plan B (rollback) ?



Ma version livrée a-t-elle un numéro et un changelog ?

Le jury évalue le pipeline entier — pas juste la démo



Un beau prototype sans méthode = un projet incomplet aux yeux du jury



Le vibe coding accélère l'étape 4 sur 8 — pas les 7 autres



Expliquer votre démarche compte autant que montrer le résultat

Bonne compétition ! Du besoin à la Release : à vous de jouer.